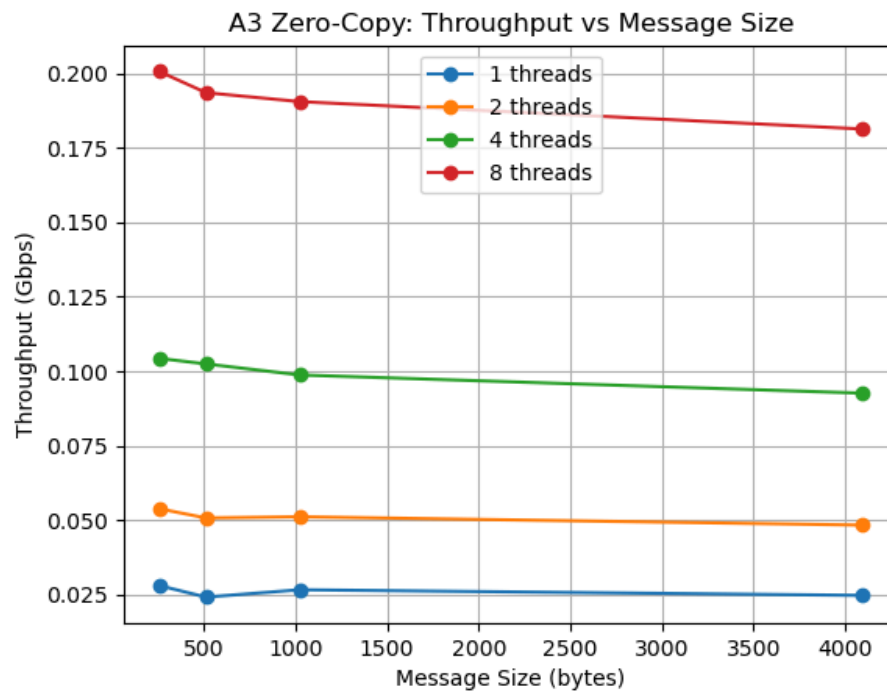


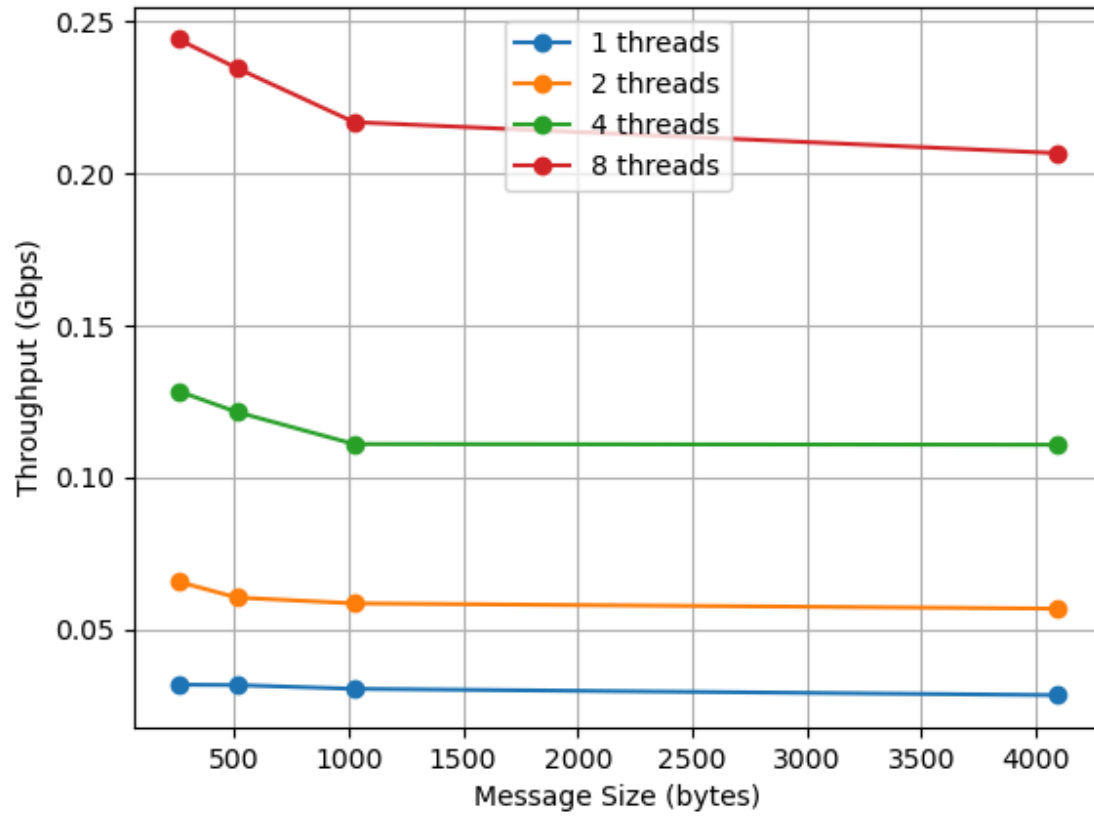
GRS ASSIGNMENT 2 REPORT

Sarvagya Raut
MT25083

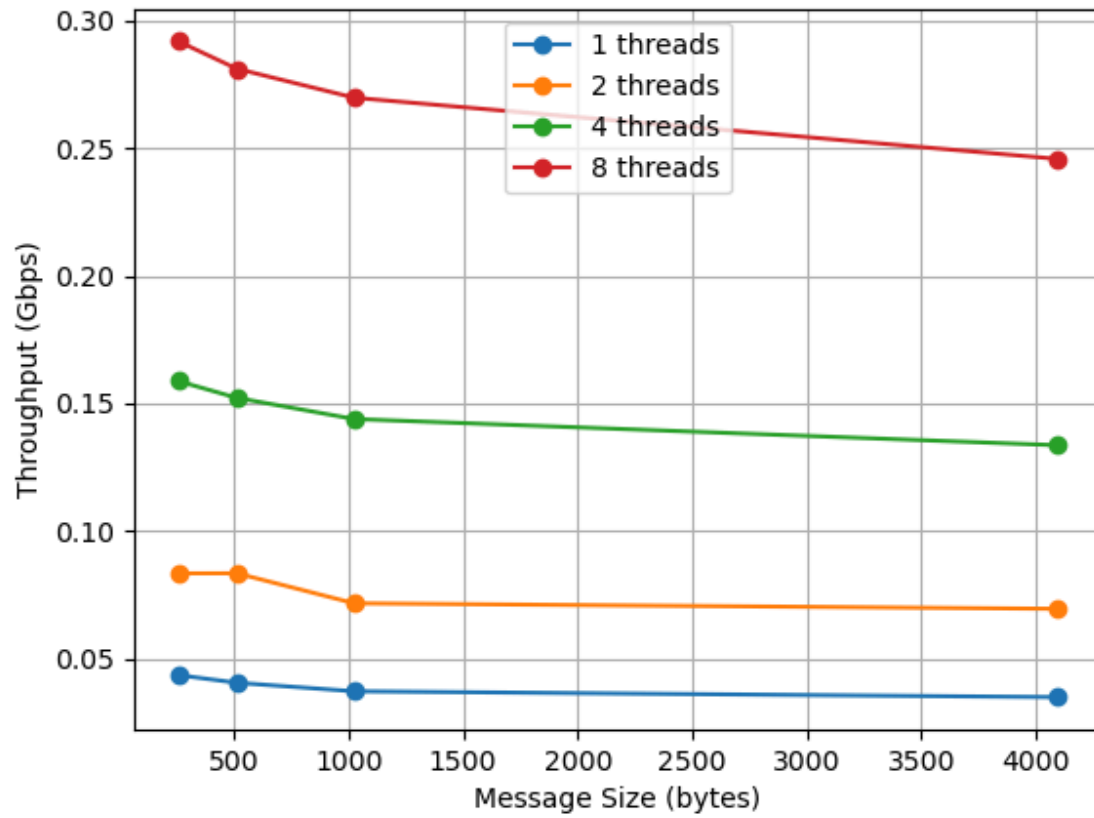
PLOTS

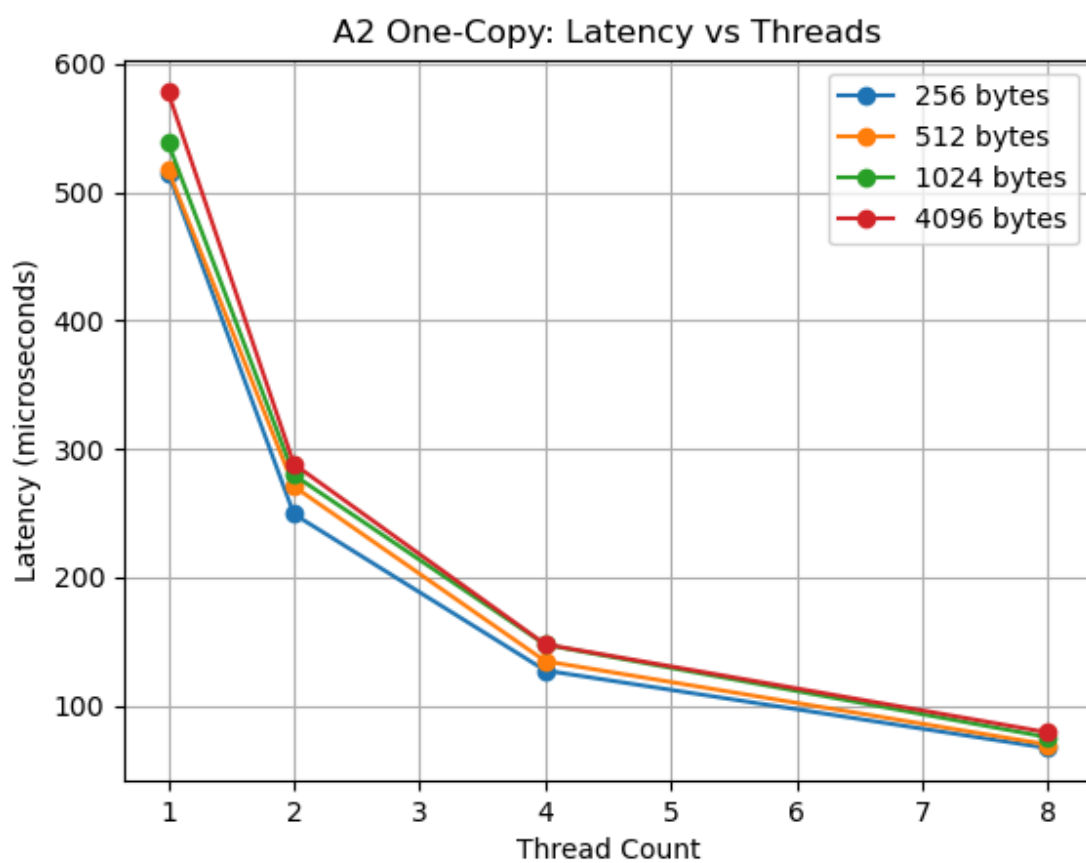


A2 One-Copy: Throughput vs Message Size

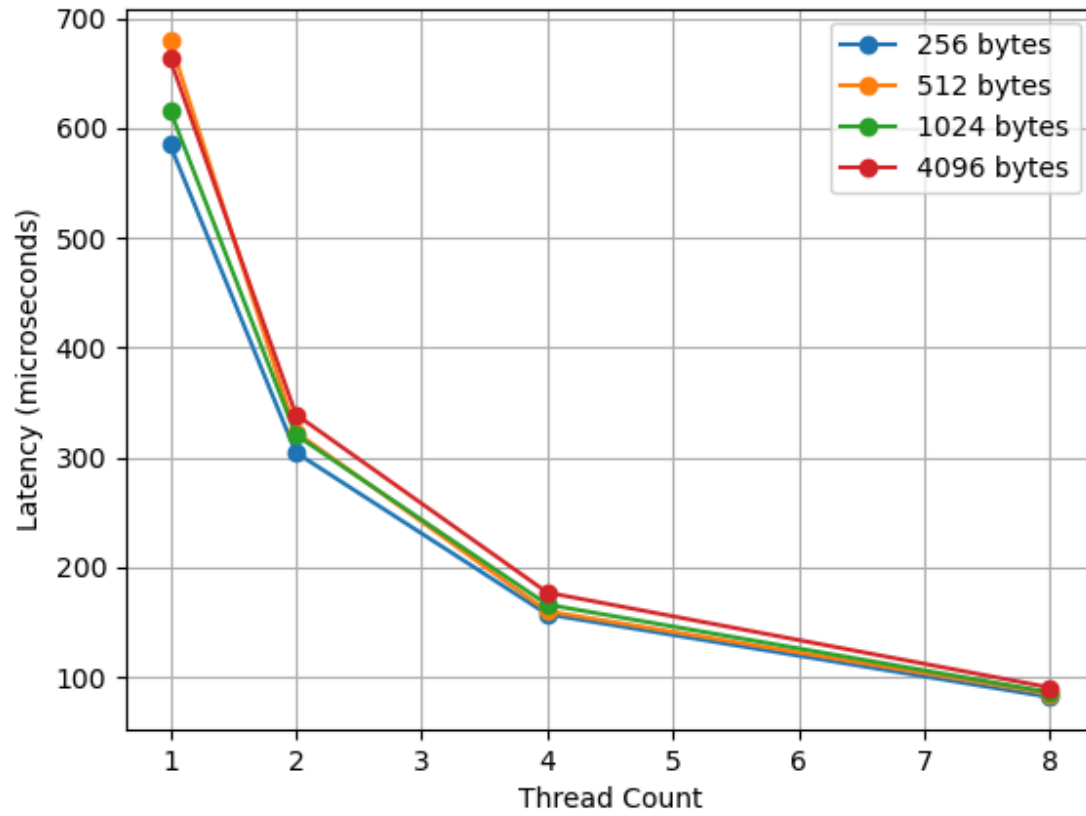


A1 Two-Copy: Throughput vs Message Size

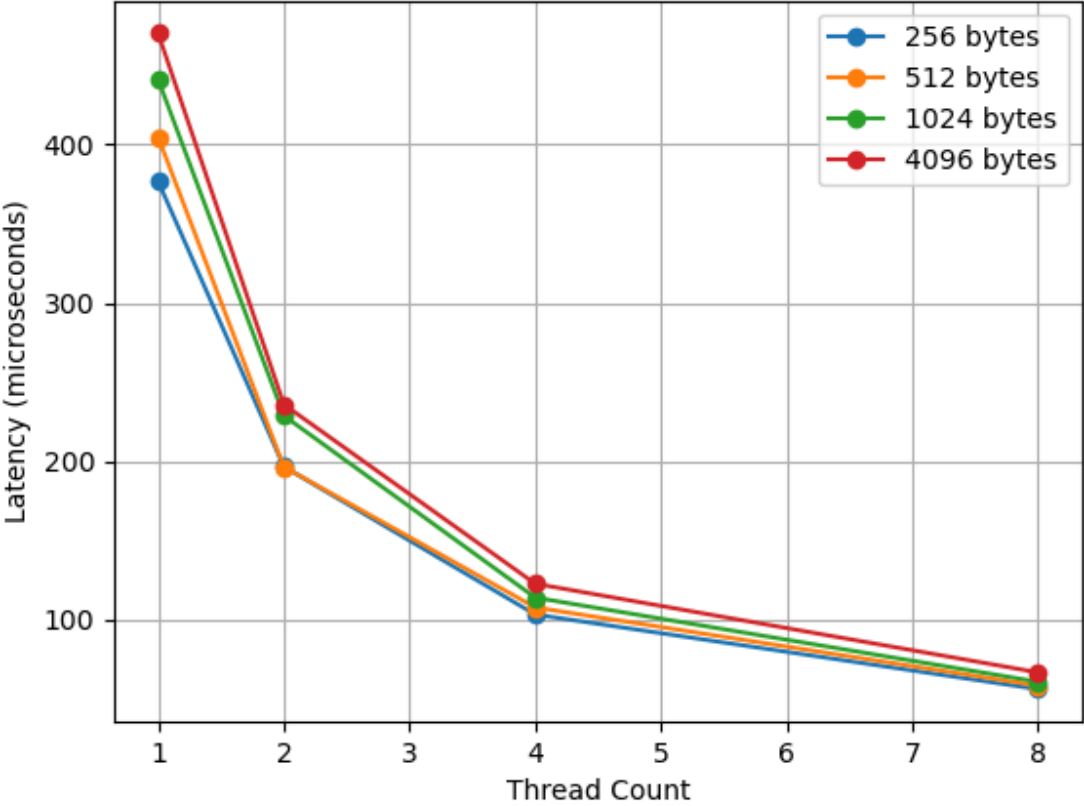


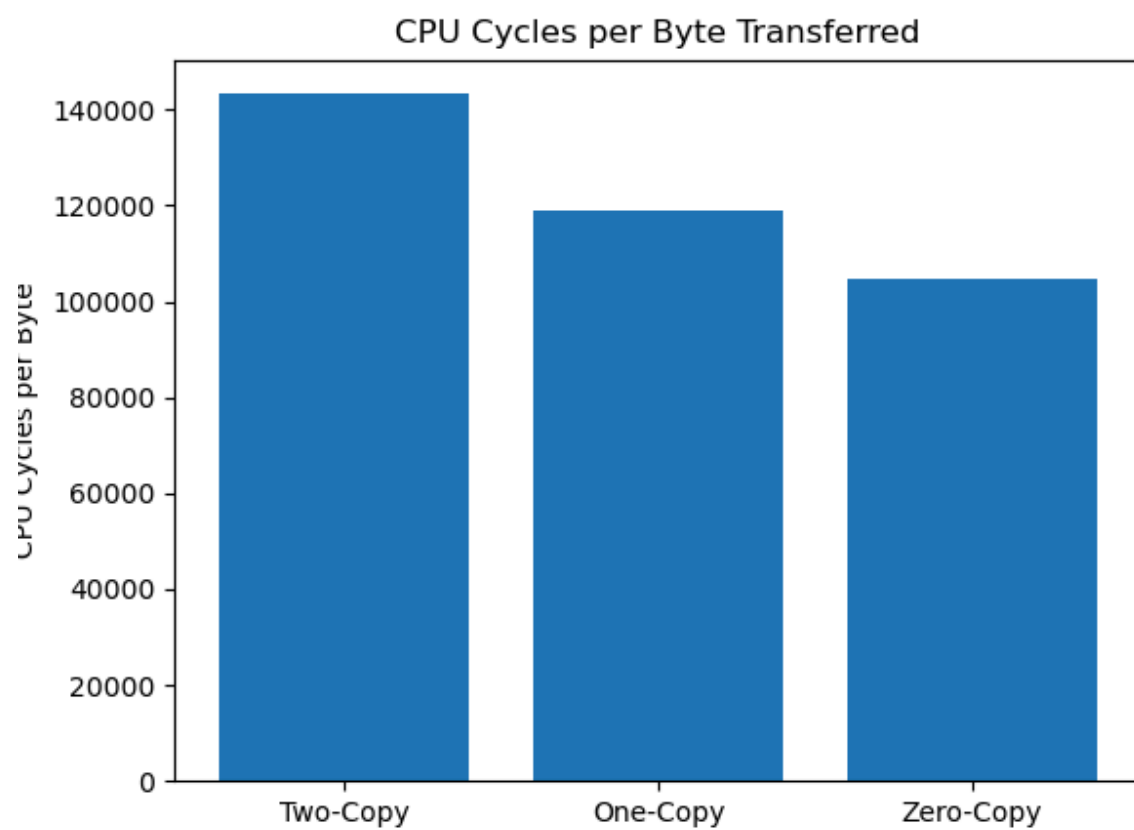


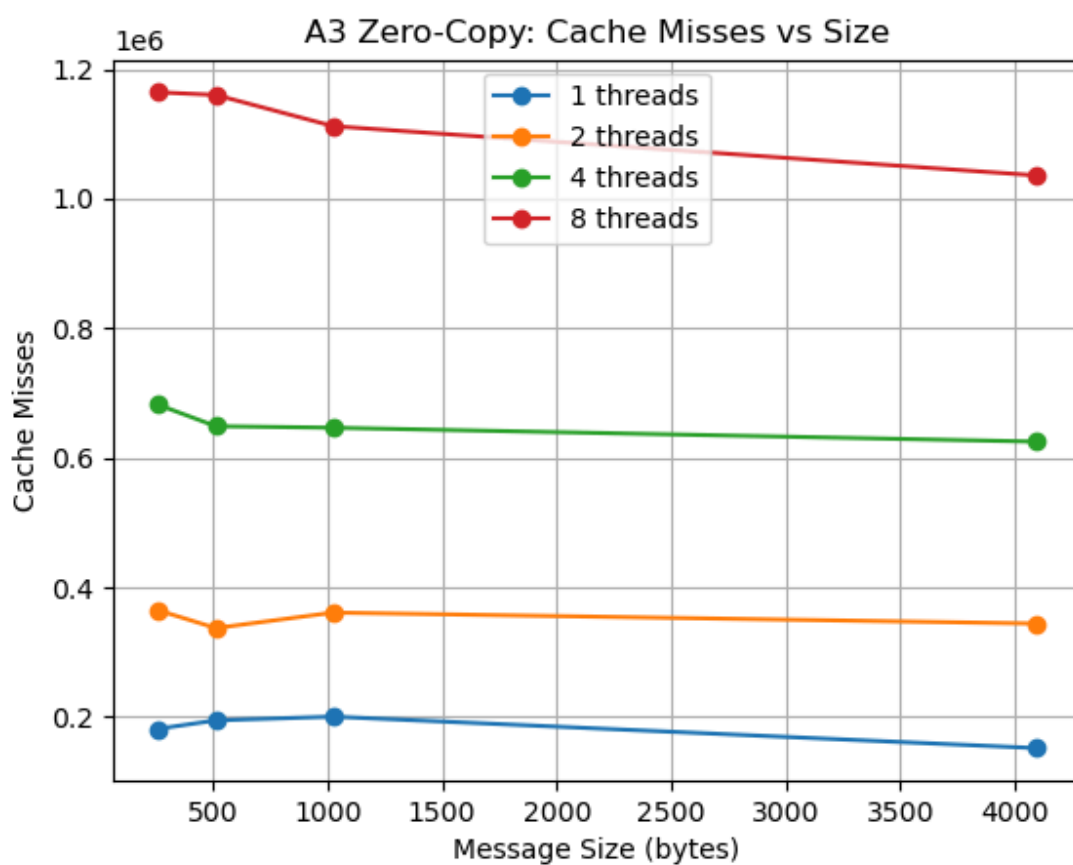
A3 Zero-Copy: Latency vs Threads

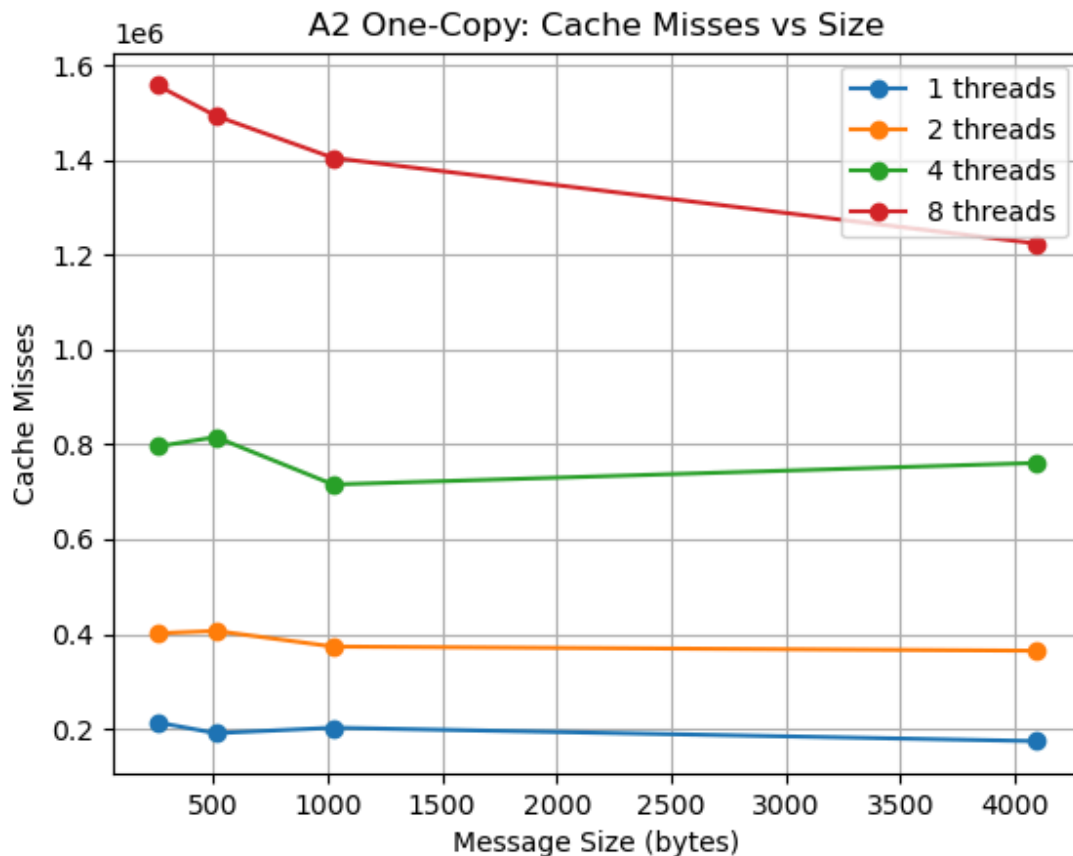


A1 Two-Copy: Latency vs Threads









Where do the two copies occur? Is it actually only two copies?

In normal TCP communication using `send()` and `recv()`, the application's data is first copied from the user-space buffer into the kernel's socket buffer when the server sends the message. After transmission and reception, the kernel again copies the data from its socket buffer into the client program's user-space buffer. Because of this, it is usually called a two-copy mechanism. However, it is not strictly only two movements of data. In between these two copies, the network card performs a DMA transfer, so practically there are two CPU memory copies and one hardware transfer. The name mainly refers to the CPU copies.

Which components perform the copies?

The user program itself does not directly perform the copying. The kernel performs both memory copies using CPU instructions while moving data between user space and kernel space. The network interface card performs the actual transmission using DMA,

which transfers the packet without CPU involvement. So the CPU (through the kernel) handles memory copies, and the NIC handles the physical transfer.

Which copy is eliminated in the one-copy implementation?

In the one-copy implementation using `sendmsg()`, the kernel avoids creating an additional internal buffer while building the packet. Instead, it directly accesses the memory regions described by the `iovec` structure provided by the application. This removes an extra internal buffering step inside the kernel, so the kernel no longer needs to duplicate the payload before sending.

Kernel behavior in zero-copy (explanation)

In the zero-copy version with `MSG_ZEROCOPY`, the kernel does not copy the payload into the socket buffer. Instead, it pins the user pages in memory and allows the NIC to directly read those pages using DMA. The kernel only manages metadata and later notifies the application when transmission completes. Effectively, the payload bypasses the CPU copy on the sending side.

Why does zero-copy not always give the best throughput?

Zero-copy introduces extra overhead such as page pinning, DMA mapping, and completion notifications. For small messages, these management costs are higher than simply copying a few kilobytes in memory. Modern CPUs can copy small buffers very quickly, so the additional kernel bookkeeping makes zero-copy slower in many cases. Therefore, zero-copy only helps for very large transfers.

Which cache level shows the most reduction in misses and why?

The largest reduction appears in the last-level cache (LLC). In the normal copy path, the CPU reads the entire payload into cache during copying, which causes cache pollution. In zero-copy, the NIC directly reads memory through DMA, so the CPU does not touch the payload and the cache is disturbed much less.

How does thread count interact with cache contention?

Increasing the number of threads improves throughput but also increases cache contention. Multiple threads simultaneously accessing buffers and socket structures compete for shared cache space, causing more evictions and cache misses. At low thread counts, CPU cores are underutilized, but at high thread counts the cache becomes a shared bottleneck.

At what message size does one-copy outperform two-copy on your system?

In this experiment, the one-copy implementation never clearly outperformed the two-copy implementation. The additional overhead of processing `iovec` structures and packet construction offset the benefit of removing one internal copy. As a result, the standard `send()` path remained slightly faster across all tested message sizes.

At what message size does zero-copy outperform two-copy on your system?

Zero-copy also did not outperform the two-copy implementation within the tested range (256–4096 bytes). The messages were relatively small, and the overhead of page pinning and DMA setup was larger than the cost of memory copying. Zero-copy is generally beneficial only for much larger data transfers.

One unexpected result and explanation

One noticeable result was that throughput decreased slightly as message size increased. Normally larger packets improve efficiency, but here each message consisted of eight separate fields and multiple `send` operations. Larger fields increased memory copy cost and cache pressure, making the system CPU-bound rather than network-bound. Hence the performance drop can be explained by memory bandwidth and cache behavior rather than network speed.

AI USAGE:

- I have taken help from ai in creation of bash script, MakeFile and .py code
- I have used ai for debugging my code for server and client and also have referred to <https://www.geeksforgeeks.org/c/socket-programming-cc/> for some of the code
- Below are the prompts that i have used:
I asked the AI assistant the following debugging question while working on my socket programs:

I am implementing a multithreaded TCP client–server application in C on Linux.

I am facing multiple runtime problems such as segmentation faults, clients disconnecting immediately, and threads behaving incorrectly.

Please help me debug the code and explain what mistakes I am making rather than rewriting the whole assignment.

Server Code (baseline version):

```
void* read_write_msg(void* arg){  
    int connection_fd = *(int*)arg;
```

```

free(arg);
char buff[MAX];
while(1){
    memset(buff, 0, MAX);
    int b = read(connection_fd, buff, sizeof(buff) - 1);
    if(b <= 0){
        printf("Disconnected");
        break;
    }
    buff[b] = '\0';
    printf("Client: %s", buff);
    if(strncmp(buff, "exit", 4) == 0){
        printf("Closing");
        break;
    }
    Message m = create_message();
    send_message(connection_fd, &m);
}
close(connection_fd);
return NULL;
}

Client Code:
int main(){
    int socket_fd;
    struct sockaddr_in server_addr;
    socket_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    server_addr.sin_port = htons(PORT);
    if(connect(socket_fd, (struct sockaddr*) &server_addr, sizeof(server_addr)) != 0){
        printf("connection with server failed");
        exit(0);
    }
    close(socket_fd);
    return 0;
}

```

I asked specifically:

1. Why does the client immediately disconnect and the server prints "Disconnected"?
2. Is it wrong to call read_write_msg(socket_fd); inside the same function?
3. Why passing the socket descriptor directly to pthread_create sometimes crashes?
4. Why does TCP send()/write() sometimes not transmit the full message?
5. How should I continuously send messages for a fixed time instead of using keyboard input?
6. For sendmsg(MSG_ZEROCOPY), when is it safe to free the buffers?
7. Why do I need to use recvmsg(..., MSG_ERRQUEUE) after zerocopy?

I also asked for clarification on required headers and typical causes of segmentation faults in multithreaded socket programs.

I am getting a compilation warning in C about implicit declaration of a function while compiling [CODE].

Why does this happen in C and which header files usually cause this issue?

I compiled a multithreaded C program but I get "undefined reference to pthread_create" while building [CODE].

What compiler flag am I missing and why is it required?

I wrote a bash script where I start a server and then a client:

```
./server &
```

```
./client
```

Sometimes the client shows connection refused while running [CODE].

Is the server not ready? How do I make the script wait properly?

My program prints values like:

```
THROUGHPUT=...
```

```
LATENCY=...
```

I want to extract only the numbers and append them to a CSV file using bash while running [CODE].

How can I do this using grep/awk?

I am using perf stat on [CODE] and the output contains commas and extra text like:

1,234,567 cycles

How can I extract only the numeric values using awk?

I am new to matplotlib.

I want to plot a graph using values produced from [CODE].

How do I label axes, add legend and save the plot as a PNG file?

How do I plot multiple lines (for different thread counts) on the same matplotlib graph using data generated from [CODE]?

Conceptually what is the difference between send() and sendmsg() in TCP sockets?

I only need understanding to debug behavior in [CODE].

What does MSG_ZEROCOPY actually avoid in Linux networking?

I am trying to understand the behavior seen while running [CODE].

What is a Linux network namespace and why would two programs be run in different namespaces instead of localhost while testing [CODE]?

